

Airlock

Project analysis · Worker, Store & Supervisor

Airlock is a web agent that does real repair, browsing, and analysis work inside disposable Kata microVMs on Vultr — while the agent never declares its own success. This document summarizes the product, architecture, maturity, and the durable orchestration spine: **Store**, **Worker**, and **Supervisor**.

CHALLENGE

Blast Radius Zero (Vultr Agent Arena)

LIVE DEMO

<https://96-30-198-46.sslip.io>

STACK

Bun · TypeScript · Hono · PGLite · React · Kata

MODEL

Vultr Serverless Inference · glm-5.3

Contents

- 1. What Airlock is
- 2. Architecture overview
- 3. Repository map
- 4. Trust model & checkpoints
- 5. Maturity & evidence
- 6. Worker, Store & Supervisor
- 7. How the three pieces fit together

1. What Airlock is

Tagline: Let the agent do anything. Nothing it does can touch you.

Airlock is built for Challenge 1 at The Agent Arena Hackathon (Vultr, San Francisco, Sep 2026): a web agent that performs real work — writing and running code, or driving a real browser — where every action is contained in a sandbox on Vultr, and a central control layer plans, dispatches, and returns verifiable output.

Two task kinds, one control plane

Kind	What it does	How success is decided
Repair	Takes an untrusted bug report for a supported library (hero: <code>tabulate-365</code> / <code>python-tabulate #365</code>), reproduces the failure, attempts a minimal repair, returns a patch.	External comparator on sealed candidate bytes in fresh sandboxes. Outcomes include <code>CANDIDATE_PASSED_CHECKS</code> , <code>CHECKS_FAILED</code> , <code>NOT_REPRODUCED</code> , etc.
General	Profiles analysis, web-research, web-analysis: Chromium, offline Python/Node, uploads, screenshots, human takeover, digest-bound form approvals.	Controller-owned completion checks on collected outputs/evidence (<code>RESULT_VERIFIED</code> / <code>RESULT_PARTIAL</code> / ...).

The agent cannot mark its own homework. A forged “all tests passed” log still fails. “Passed these checks” means exactly the frozen cases passed — not safe, certified, or generally correct.

2. Architecture overview

Two Vultr VMs in region `atl`, linked by VPC:

Plane	Where	Owns	Never
Control apps/control, apps/web	VM A · Cloud Compute	Identities, contracts, phases, budgets, sessions, export, Vultr model client, comparator, artifacts	Holds Docker socket; runs candidate code; trusts sandbox “passed” fields
Supervisor apps/supervisor	VM B · VX1 (/dev/kvm)	Container identity, execution, deadlines, termination, cleanup, isolation checks	Accepts caller-chosen image, mount, network, runtime, or host path
Comparator	Control plane	Required case IDs, expected behaviour, verdict	Imports or executes candidate code
Model	Vultr Serverless Inference	Proposes edits and tool calls	Declares success, grants privileges, publishes

Repair loop (phases)

prepare → baseline → reproduce → repair → freeze → verify → ready

- **Baseline / verify:** one fresh one-shot sandbox per contract case.
- **Freeze:** revoke → stop → collect allowed files → seal SourceManifest.
- **submit_candidate:** only advances to freeze; does not mean pass.

Reuse

Adapted modules from OpenMuse (worker, store patterns) and OpenBot (narrow supervisor). Neither upstream app is the composition root; CopilotKit / AG-UI are not dependencies. Pins and modifications are recorded in THIRD_PARTY_NOTICES.md.

3. Repository map

apps/control	API, RepairHandler, GeneralHandler, verifier, store, worker
apps/supervisor	Narrow Docker/Kata service (execution plane)
apps/web	Repair + general UI, evidence, checkpoints, approvals
apps/egress	Per-attempt browser egress proxy (allowlist, DNS pin)
apps/fixtures	Demo data / airlock-forms-v1 destination
packages/contracts	Shared Zod schemas at every boundary
profiles/	Supported repair cases (tabulate-365)
runtime/	Pinned Python, browser, analysis, node images + collectors
deploy/	Vultr provision + host setup scripts
docs/	Status ledger, evidence, runbook, decisions
research/	Design history (35 / 37 / 38 / 40 / 42)

Stack: Bun workspaces, TypeScript (strict), Hono, PGLite, dockerode, React/Vite, Playwright/Chromium, Python 3.12 pinned runtime image.

4. Trust model & checkpoints

Standing invariants (CLAUDE.md §3)

1. **One authority per concern** — controller, supervisor, comparator, artifacts; model owns none.

2. **Untrusted code only in its sandbox** — report text, generated code, candidate library never run on VM A.
3. **Candidate identity is immutable** — verify / preview / export share one sealed digest.
4. **Worker cannot declare success** — sandbox exit codes and forged logs carry no authority.
5. **Cancellation has an execution effect** — revoke → stop → confirm → fence late results.
6. **Ownership checked at every access** — IDs are not bearer tokens.
7. **No secrets / network / host in code sandboxes** — browser reaches only its allowlisting egress proxy.
8. **Runtime tier inspected, never assumed** — Kata target; gVisor floor; runc is dev-unsafe only.
9. **Five checkpoints on every run record.**
0. **All runtime model calls go through Vultr Serverless Inference.**

The five checkpoints

#	Checkpoint	Proves
1	Host check	CPU virt, /dev/kvm, runtimes, selected runtime, not dev-unsafe
2	Execution log	Every command with exit code, duration, bounded output
3	In-sandbox identity	hostname + uname from inside (Kata guest ≠/host)
4	Isolation probe	metadata · DNS · TCP · Docker socket · mounts all BLOCKED before agent work
5	Teardown	Host-wide listing reads "(no sandboxes)"

5. Maturity & evidence

Gate	Result	Notes
Preflight on VX1	49/49	KVM, Kata, hardening, probe, quota, Chromium under Kata, egress, empty host
Live repair gate	3/3	Fresh tabulate #365 repairs through public URL, comparator-judged
Vultr acceptance	9/9	Analysis, research, repair+export, hero, approvals, takeover, containment
Findings ledger	~95 items	Nearly all closed (1 conditional, NetBird optional)

Evidence lives under docs/evidence/. Demo video: docs/demo/vultr-demo.mp4. Documented limitations include: GET to allowlisted hosts can carry data; pages needing web workers may not run; RESULT_VERIFIED is structural; single control-plane process.

6. Worker, Store & Supervisor

These three pieces form the durable orchestration spine. The **Store** remembers; the **Worker** drives tasks from that memory; the **Supervisor** is the only place that actually executes.

```

UI / API → Store (PGLite on VM A)
          ↑↓
          TaskWorker → RepairHandler / GeneralHandler
                      ↓ journaled HTTP over VPC
                      Supervisor (VM B, Docker/Kata)

```

The model and handlers never touch Docker. The supervisor never owns task identity or outcomes.

6.1 Store — apps/control/src/store

Control-plane database on **VM A**, adapted from OpenMuse's `db.ts`, running **PGLite** (Postgres-compatible, in-process).

TABLES

Table	Role
records	Generic JSON documents keyed by (owner, kind, id) — tasks, manifests, grants, worker status
events	Append-only run log with per-task monotonic seq (SSE replay via Last-Event-ID)

KEY WRITE PATHS

- `put` — upsert (mutable things like task progress).
- `compareAndSwap` — atomic claim/update only if expected fields match (leases, approvals).
- `insertImmutable` — insert-or-refuse; used for verification records, manifests, export grants (no overwrite).
- `appendEvent` / `listEvents` — durable, ordered event stream.
- `scanWhere` — worker polling by status without loading every record.

Ownership is first-class: every record is scoped to an owner. Task IDs are not bearer tokens; cross-owner access reads as 404. Generic upsert is forbidden for artifacts — immutable identity is a trust invariant.

6.2 Worker — apps/control/src/worker

`TaskWorker` is the durable lease loop on **VM A**, adapted from OpenMuse's `engine/worker.ts`. It does *not* know how to repair or browse — it only claims work and calls a `TaskHandler`.

TICK CYCLE (~1S)

1. Scan Store for queued, expired-lease running, or cancelling tasks.
2. **CAS-claim** with a fresh `leaseId` + `leaseUntil`.
3. Invoke the handler with a `TaskContext`:
 - `mode`: "run" or "cancel"
 - `guard()` / `checkpoint()` — throw `LostLeaseError` if the lease is gone
 - `event()` — append to Store's event log
4. Heartbeat renews the lease while the handler runs.
5. On finish: clear lease, apply the handler's terminal patch.

HANDLERS PLUGGED AT STARTUP

Handler	Role
RepairHandler	Deterministic phases: prepare → baseline → reproduce → repair → freeze → verify → ready
GeneralHandler	Browser/code tools + controller-owned completion checks

CANCELLATION

Not a DB flag alone. The API sets `cancelling` → worker runs `cancel mode` → `revoke/stop/confirm` on the supervisor → only then `cancelled`. Unconfirmed teardown stays `cancelling` and retries (bounded); past the bound the task is recorded `failed` with an honest unconfirmed teardown — never a forged `cancelled`.

6.3 Supervisor — apps/supervisor

Narrow HTTP service on **VM B** — the **only** process with the Docker socket. Forked from OpenBot's supervisor idea, extended with execution, freeze, invoke, fencing, capacity, and janitor.

THE REAL BOUNDARY

A bearer token keeps strangers off the port. The **API vocabulary** is the wall: callers can say `create attempt / run tool / freeze / revoke / destroy` — never “use this image / mount / network / runtime / host path,” because those are not expressible in the API.

LOCAL JOURNAL (SQLITE ON VM B)

Separate from the control Store. Guarantees:

- Operation id + request digest → replay the same result, or conflict if digest differs.
- Pending ops after a crash become `interrupted`, never a forged receipt.
- **Generation fencing** — stale or revoked generations get 409 before every exec.

LIFECYCLE

`create` → `inspect` `hardening` → `isolation probe` → `tools` → `freeze` → `destroy`

Freeze ordering: `revoke` → provision collector with volume read-only → `stop author` → settle outstanding execs → collect. Host **capacity admission** returns 429 before any Docker call if the budget is full.

MAIN ROUTES

Route	Purpose
POST /attempts	Create disposable attempt (author / hostile / ...)
POST .../tool	Bounded read / write / exec inside the attempt
POST .../renew	Extend renewable execution authorization
POST .../freeze	Stop and collect sealed file envelope
POST .../revoke · /destroy	Stop authority; remove container + volume
POST /invoke	Fresh one-shot verify/baseline/preview sandbox
POST /hostile	Containment demo (rm -rf / fork bomb) with survival card
GET /host · /capacity · /listing	Host check, admission budget, host-wide inventory

SANDBOX ROLES

author, one-shot invoke roles (baseline / candidate / preview), hostile, analysis, node, plus browser + egress pair. Code roles use `--network none`; the browser reaches only its per-attempt egress proxy.

7. How the three pieces fit together

Piece	Owns	Never
Store	Task identity, leases, events, sealed artifacts on VM A	Execution or Docker
Worker	Claiming, heartbeats, cancel retries, calling handlers	Declaring pass/fail or creating containers
Supervisor	Containers, deadlines, isolation, cleanup on VX1	Trusting caller-chosen mounts/images, or deciding task outcomes

Handlers (Repair / General) sit between Worker and Supervisor: they read and write the Store through the worker context, and every supervisor mutation is journaled on the control side *before* the HTTP call goes out. Crashes surface as unknown / interrupted, never as invented receipts.

End of briefing. For day-to-day development rules see `CLAUDE.md`; for the findings ledger see `docs/implementation-status.md`; for deployment see `README.md` and `docs/runbook.md`.